

Digital Wallet System Report

Name – Devpura Dhruvin Vijayprakash

Reg. No. – 25BAI10276

Branch – CSE (Artificial Intelligence and Machine Learning)

Year – Second Year

Course – Programming in Java – CSE2006

Date of Submission – 18 September 2026

University – VIT Bhopal University

Table of Contents

1. Introduction
2. Problem Statement
3. Functional Requirements
4. Non-functional Requirements
5. System Architecture
6. Design Diagrams
7. Design Decisions & Rationale
8. Implementation Details
9. Screenshots / Results
10. Testing Approach
11. Challenges Faced
12. Learnings & Key Takeaways
13. Future Enhancements
14. References

1. Introduction

The Digital Wallet System is a console-based Java application that I built to mimic the functions of a digital wallet for learning purposes. The Digital Wallet System offers a menu that lets users register log in with a PIN create a wallet manage balances transfer money add beneficiaries, pay bills, view transaction history and export wallet statements.

I designed the Digital Wallet System to be a hands-on example of Java programming. The Digital Wallet System uses classes to represent users, wallets, transactions, beneficiaries, bill payments and payment methods. File I/O keeps data while SQLite and JDBC handle database tasks. I also added a synchronized wallet demo to show how multithreading and synchronization work.

The Digital Wallet System is a simulation meant for teaching. The Digital Wallet System does not connect to banking networks, payment gateways or utility companies. The goal of the Digital Wallet System is to show a Java application workflow, with clear input, processing, validation, persistence and output.

2. Problem Statement

Managing digital wallet activities through separate systems can make basic account and transaction management harder to understand and track.

For a project there is a need for a simple system that combines user management, wallet operations, transfers, bill payment and transaction reporting, within one application.

The Digital Wallet System addresses this problem by providing a console application where a registered user can log in create a wallet perform supported wallet operations and review transaction records. The project also demonstrates how the individual Java programming concepts taught in the course can be combined into a business-oriented application.

3. Functional Requirements

The system provides the following functional modules and capabilities:

Module	Requirement
User Management	Register a user, validate details, and log in using email and a 4-digit PIN.
Wallet Management	Create a wallet, check balance, add money, and withdraw money.
Money Transfer	Send money to another registered user with an active wallet.
Beneficiary Management	Add, view, and remove saved beneficiaries.
Bill Payment	Simulate electricity, water, mobile, internet, and gas bill payments.
Transaction Management	Record add-money, withdrawal, transfer, receive, and bill-payment transactions.
Statement Generation	Export the wallet balance and transaction history to a text file.
Database Operations	Initialize the SQLite database, store user information with JDBC, and read stored users.
Concurrency Demonstration	Run synchronized balance operations using multiple threads in WalletConcurrencyDemo.

4. Non-functional Requirements

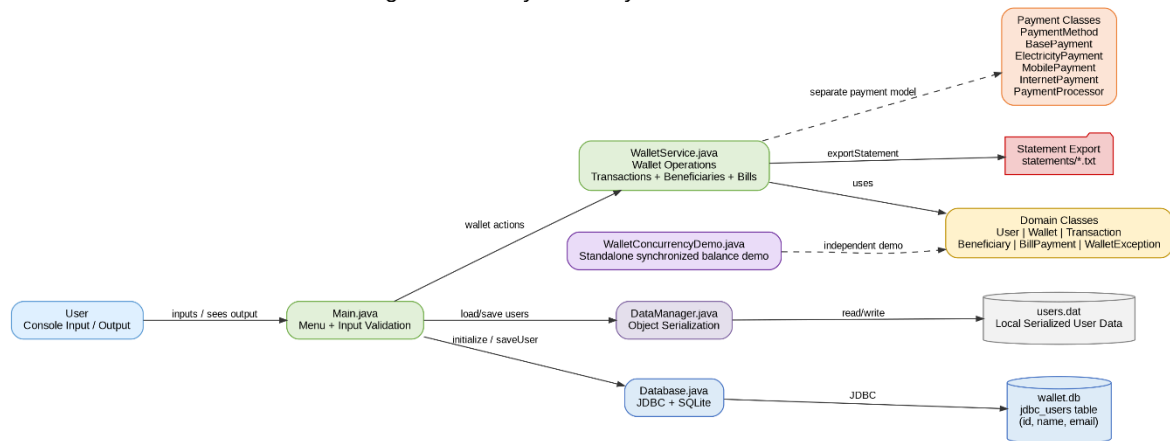
Requirement	How the system addresses it
Usability	Provide a simple menu-driven console interface with clear prompts and feedback.
Reliability	Prevent invalid transactions such as negative amounts and withdrawals beyond the available balance.
Security	Use a 4-digit PIN for application login and avoid database usernames/passwords.
Maintainability	Separate responsibilities across multiple focused Java classes.
Resource Efficiency	Use lightweight local serialization and SQLite storage for the educational application.
Error Handling	Use input validation and the custom WalletException class for wallet-related errors.

5. System Architecture

The system follows a simple layered structure appropriate for a console-based Java application. The Main class acts as the entry point and controls the user interaction flow. Domain classes hold application data, service classes perform wallet operations, persistence classes save or retrieve information, and payment classes demonstrate abstraction and polymorphism.

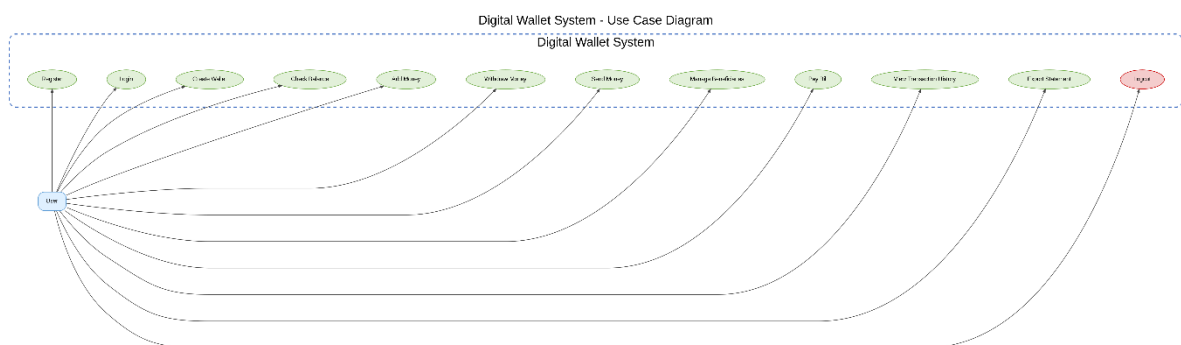
Layer	Main components	Responsibility
Presentation / Control	Main	Displays menus, reads input, validates user data, and directs application flow.
Domain Model	User, Wallet, Transaction, Beneficiary, BillPayment	Represents users, wallet state, transaction records, beneficiaries, and bill-payment details.
Service	WalletService	Performs money operations, transfers, bill payments, beneficiary operations, and statement export.
Persistence	DataManager, Database	Uses object serialization and SQLite/JDBC for data persistence.
Payment / Concurrency	PaymentMethod, BasePayment, payment classes, PaymentProcessor, WalletConcurrencyDemo	Demonstrates interfaces, inheritance, polymorphism, and synchronized multithreading.

Digital Wallet System - System Architecture



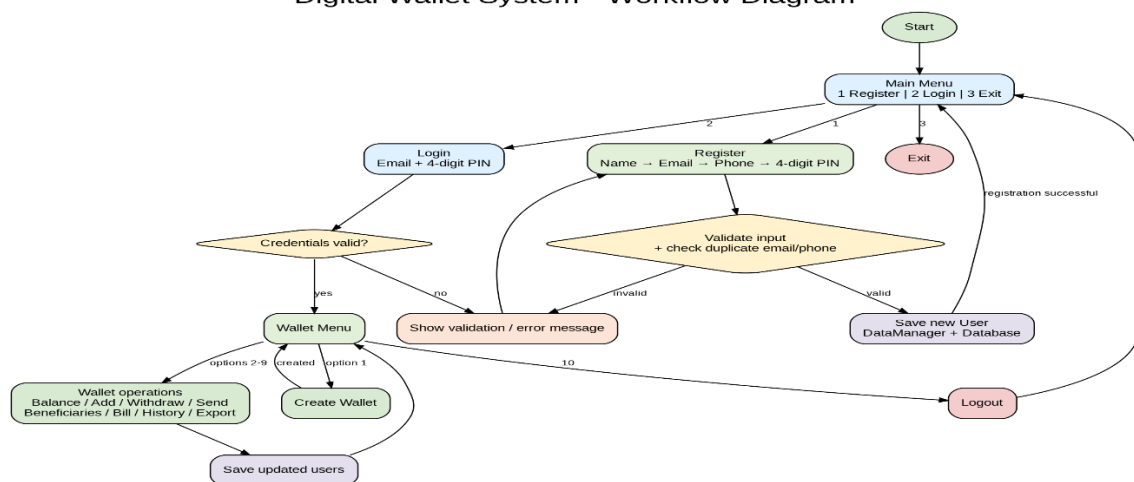
6. Design Diagrams

6.1 Use Case Diagram

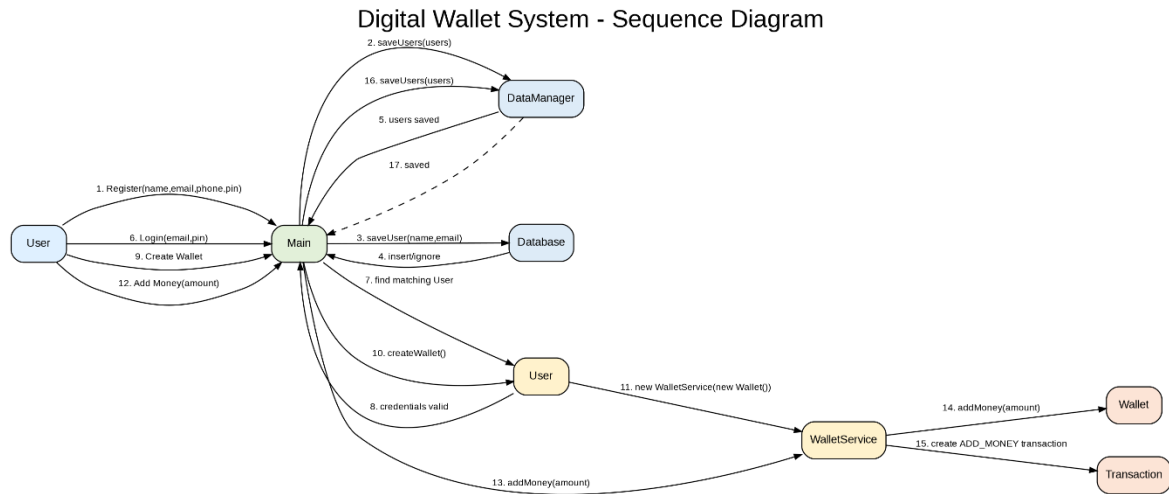


6.2 Workflow Diagram

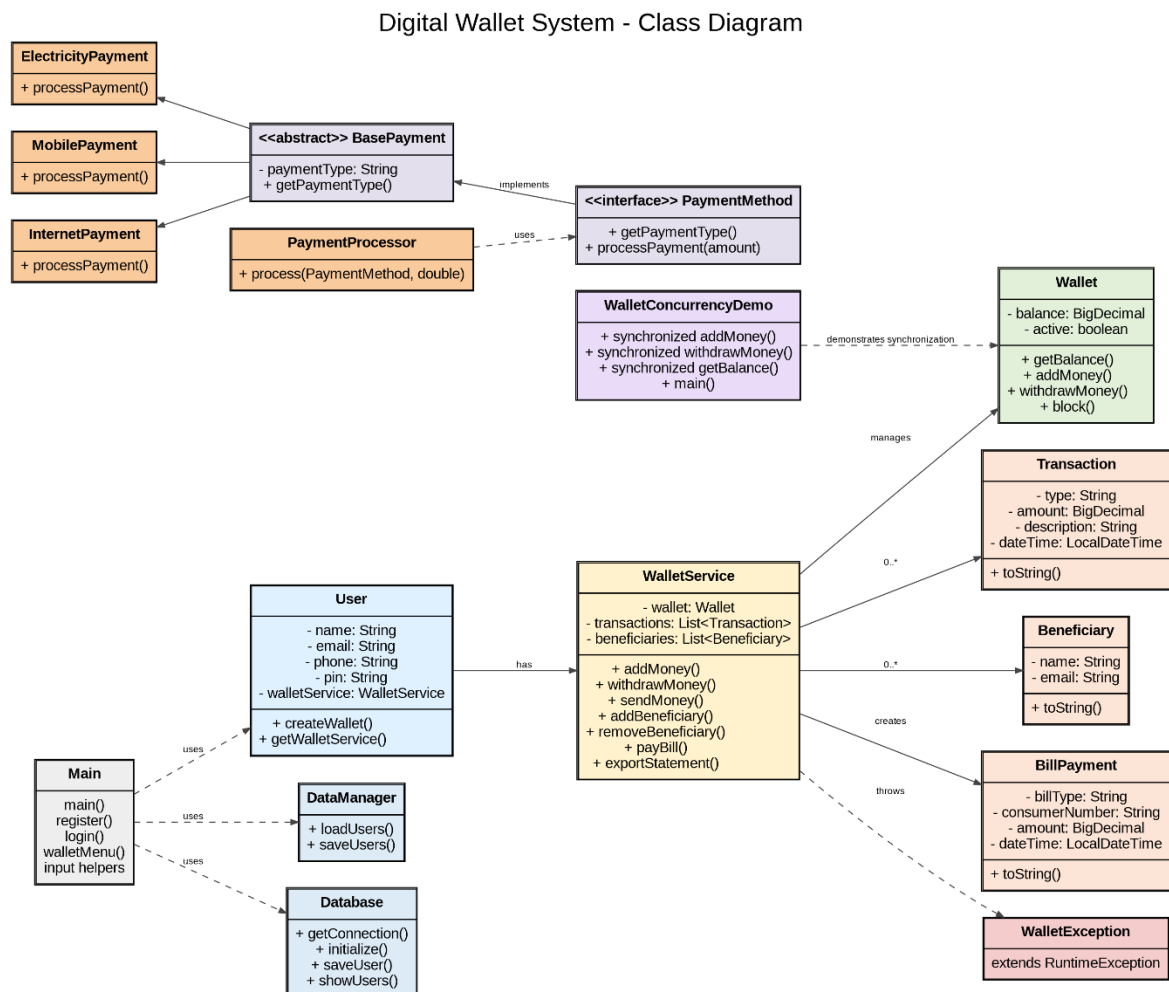
Digital Wallet System - Workflow Diagram



6.3 Sequence Diagram

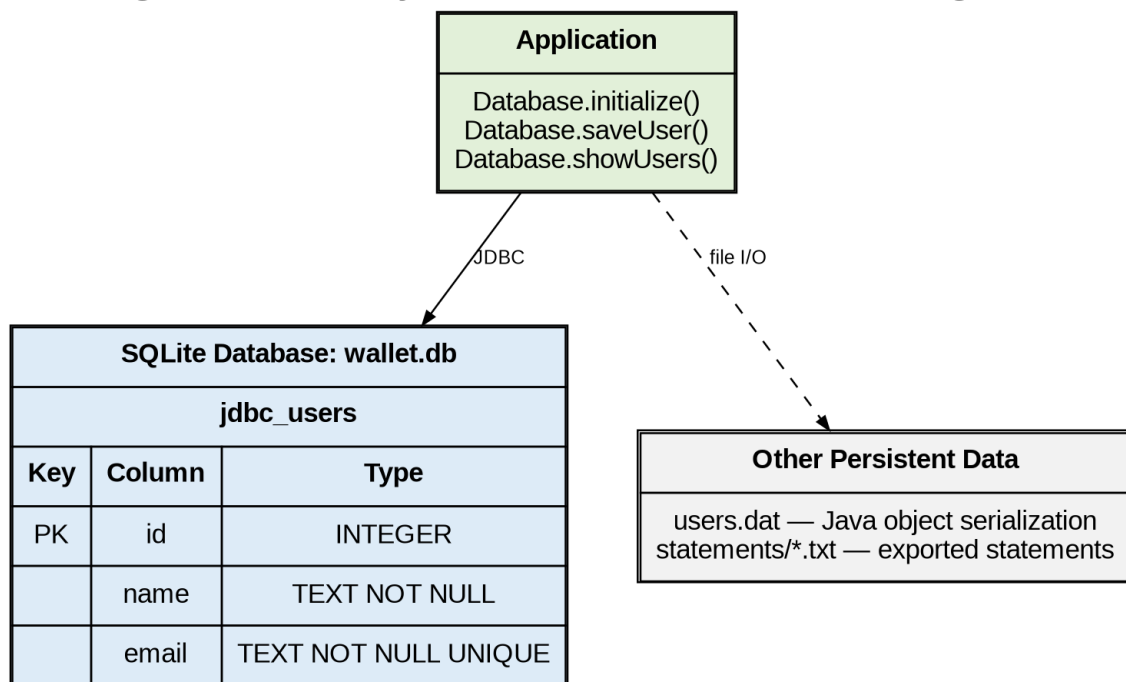


6.4 Class Diagram



6.5 ER Diagram

Digital Wallet System - Database / ER Diagram



6.6 Schema Design

The SQLite database used by the application contains the following relational table:

Table	Column	Type / Constraint	Purpose
jdbc_users	id	INTEGER, PRIMARY KEY, AUTOINCREMENT	Unique database identifier for a stored user.
jdbc_users	name	TEXT, NOT NULL	Stores the user name.
jdbc_users	email	TEXT, NOT NULL, UNIQUE	Stores the user email and prevents duplicate database entries.

7. Design Decisions & Rationale

Decision	Rationale
Console-based interface	Keeps the project focused on Java programming concepts and makes the application easy to execute through a command-line or IDE run configuration.
Separate domain classes	Keeps data and responsibilities organized and demonstrates object-oriented design.
WalletService for business logic	Centralizes wallet operations instead of placing transaction rules directly in the user interface.
BigDecimal for monetary values	Provides appropriate decimal arithmetic for money calculations without relying on floating-point representation.
Custom WalletException	Provides a single application-specific way to report wallet operation errors.
File serialization	Provides simple local persistence for the application-level user and wallet data.
SQLite with JDBC	Provides a lightweight local relational database and demonstrates JDBC connection, SQL execution, prepared statements, and result-set handling.
PaymentMethod interface + BasePayment	Provides a practical example of abstraction, inheritance, and polymorphism.
Synchronized concurrency demo	Demonstrates safe access to shared wallet balance updates when multiple threads operate on the same object.

8. Implementation Details

8.1 User Management

Main collects a user name, email, phone number, and 4-digit PIN during registration. Duplicate email addresses and phone numbers are checked before a new User object is added. Login searches the loaded users and checks the email and PIN before opening the wallet menu.

8.2 Wallet Operations

Wallet stores the current balance and active state. WalletService validates transaction amounts, checks wallet status, and updates the wallet. Successful operations create Transaction objects so that the history can be displayed later.

8.3 Money Transfer

The transfer operation locates the receiver by email, checks that the receiver is different from the sender and has a wallet, validates the amount, deducts funds from the sender, adds funds to the receiver, and records transfer and receive transactions.

8.4 Beneficiaries and Bill Payments

Beneficiaries are stored in a list associated with the wallet service and can be added, displayed, or removed. Bill payment accepts a selected bill type, consumer number, and amount. The amount is deducted from the wallet and a corresponding transaction record is created.

8.5 Persistence and JDBC

DataManager uses Java object serialization to save and load the list of users in users.dat. Database uses JDBC with the SQLite driver to create the jdbc_users table, save user records using a prepared statement, and read stored users using a ResultSet.

8.6 Statement Export

WalletService creates a statements directory and writes a text statement containing the customer name, current balance, and transaction history. The exported statement uses UTF-8 text output.

8.7 Payment Processing

PaymentMethod defines the payment contract. BasePayment stores the common payment type. ElectricityPayment, MobilePayment, and InternetPayment extend BasePayment and implement processPayment. PaymentProcessor accepts a PaymentMethod reference and therefore demonstrates runtime polymorphism.

8.8 Multithreading

WalletConcurrencyDemo creates two threads that modify a shared balance. The add and withdrawal methods are synchronized, and join() is used so that the main thread waits for both worker threads to finish.

9. Screenshots

9.1 Registration

```
1. Register
2. Login
3. Exit
Enter choice: 1

===== REGISTER =====
Enter full name: Dhruv Sharma
Enter email: dhruv@gmail.com
Enter phone number: 9898727211
Create a 4-digit PIN: 1234
```

9.2 Wallet Menu

```
===== WALLET MENU =====
1. Create Wallet
2. Check Balance
3. Add Money
4. Withdraw Money
5. Send Money
6. Beneficiaries
7. Pay Bill
8. Transaction History
9. Export Statement
10. Logout
Enter choice: 
```

9.3 Transaction History

```
===== WALLET MENU =====
1. Create Wallet
2. Check Balance
3. Add Money
4. Withdraw Money
5. Send Money
6. Beneficiaries
7. Pay Bill
8. Transaction History
9. Export Statement
10. Logout
Enter choice: 8

===== TRANSACTION HISTORY =====
2026-09-17T22:13:04.014805900 | ADD_MONEY | Rs.10000 | Money added to wallet
2026-09-17T22:13:24.362431900 | WITHDRAW | Rs.200 | Money withdrawn from wallet
2026-09-17T22:14:14.431943600 | TRANSFER | Rs.100 | Money sent
2026-09-17T22:16:12.958193100 | BILL_PAYMENT | Rs.10 | Electricity bill payment
```

10. Testing Approach

Test case	Action	Expected / observed result
Registration	Valid name, email, phone, and 4-digit PIN	Account created and registration confirmation displayed.
Email / name validation	Invalid name or email format	Validation message displayed and input requested again.
Phone / PIN validation	Invalid phone number or PIN length	Validation message displayed and input requested again.
Wallet creation	Create wallet, then attempt to create again	Wallet is created once; repeated creation reports that it already exists.
Add money	Positive valid amount	Balance increases and ADD_MONEY transaction is recorded.

Withdraw money	Amount within balance and amount above balance	Valid withdrawal succeeds; excessive withdrawal is rejected.
Money transfer	Send money to another user with a wallet	Sender balance decreases, receiver balance increases, and transfer/receive records are created.
Beneficiaries	Add, view, and remove beneficiary	Beneficiary list changes as expected.
Bill payment	Select a bill type and pay a valid amount	Payment details are displayed and BILL_PAYMENT transaction is recorded.
Transaction history	Perform one or more transactions and view history	Recorded transactions are displayed.
Statement export	Export after at least one transaction	Statement text file is created in the statements folder.
Data persistence	Exit and restart after registration/wallet activity	Previously saved application data remains available after restart.

11. Challenges Faced

- Keeping wallet operations and transaction rules separate from the console menu logic while preserving a simple command-line workflow.
- Implementing persistent application data so that users and wallet state remain available after restarting the application.
- Handling input validation for names, emails, phone numbers, PINs, menu choices, and monetary values.
- Using SQLite through JDBC while keeping the database local and avoiding external database credentials.
- Ensuring synchronized operations in the multithreading demonstration correctly protect a shared wallet balance.
- Organizing the final project repository so that source code, screenshots, diagrams, report files, and generated runtime files are handled separately.

12. Learnings & Key Takeaways

- Applied encapsulation, abstraction, inheritance, interfaces, and polymorphism in a practical business application.
- Learned to separate user-interface flow from service and data responsibilities.
- Practised Java Collections through lists of users, transactions, and beneficiaries.
- Applied file I/O and object serialization for application-level persistence.
- Used JDBC concepts including connections, SQL statements, prepared statements, and ResultSet processing.
- Applied exception handling and input validation to prevent invalid wallet operations.
- Practised multithreading and synchronization through a shared-balance example.
- Learned how Git and GitHub can be used for source-code version control and project documentation.

13. Future Enhancements

- Add a graphical or web-based interface while keeping the same service layer.
- Use a production-grade relational database for larger-scale deployment.
- Add stronger authentication such as OTP or biometric verification.
- Integrate real payment gateways and utility providers in a production implementation.
- Add administrative monitoring, audit logs, and more detailed reporting.
- Add automated unit and integration tests for the service layer and validation rules.
- Support multiple currencies and more configurable wallet features.

14. References

- VITyarthi - Build Your Own Project: General Project Instructions & Submission Guidelines, provided for the course project.
- Oracle Java Documentation - Java Language and API documentation.
- Xerial SQLite JDBC Driver documentation and project information.
- Java JDBC API documentation for database connectivity and SQL processing.
- CSE2006 Programming in Java course syllabus and learning material.